

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import sqlite3
from scipy.stats import ttest_ind
import scipy.stats as stats
warnings.filterwarnings('ignore')

```

Loading the dataset

```

# creating database connection
conn = sqlite3.connect('inventory.db')

# fetching vendor summary data
df = pd.read_sql_query('select * from vendor_sales_summary', conn)
df.head()

```

	VendorNumber	VendorName	Brand	Description	PurchasePrice	Ac
0	1128	BROWN-FORMAN CORP	1233	Jack Daniels No 7 Black	26.27	36.
1	4425	MARTIGNETTI COMPANIES	3405	Tito's Handmade Vodka	23.19	28.
2	17035	PERNOD RICARD USA	8068	Absolut 80 Proof	18.24	24.
3	3960	DIAGEO NORTH AMERICA INC	4261	Capt Morgan Spiced Rum	16.17	22.
4	3960	DIAGEO NORTH AMERICA INC	3545	Ketel One Vodka	21.89	29.

Exploratory Data Analysis

- Previously, we examined the various tables in the database to identify key variables, understand their relationships, and determine which ones should be included in the final analysis.
- In this phase of EDA, we will analyze the resultant table to gain insights into the distribution of each column. This will help us understand data patterns, identify anomalies, and ensure data quality before proceeding with further analysis.

```

# summary statistics
df.describe().T

```

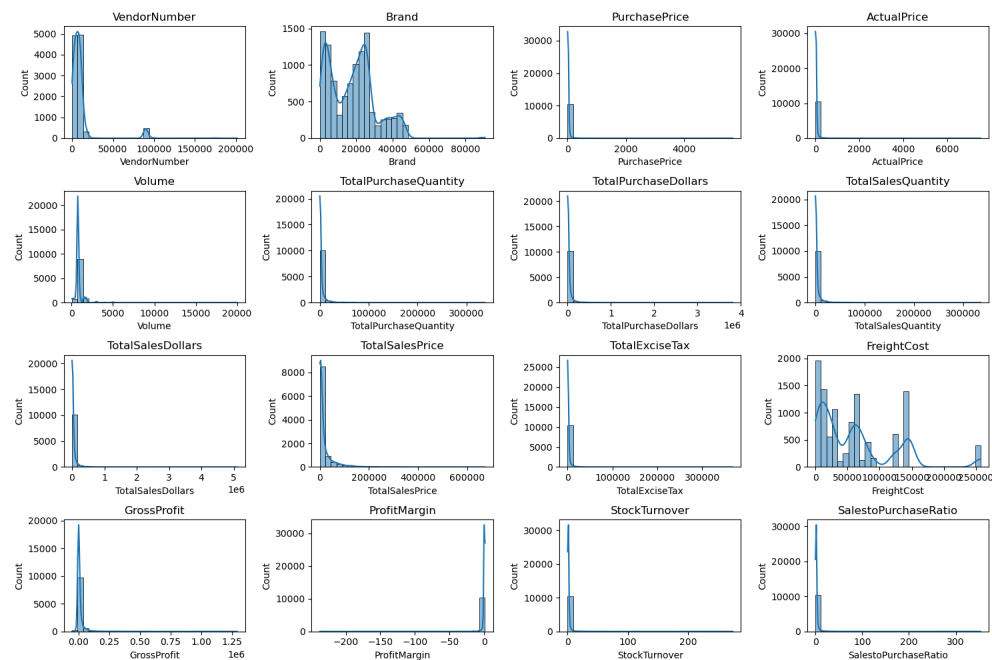
	count	mean	std	min	
VendorNumber	10692.0	1.065065e+04	18753.519148	2.00	39

	count	mean	std	min	max
Brand	10692.0	1.803923e+04	12662.187074	58.00	57100
PurchasePrice	10692.0	2.438530e+01	109.269375	0.36	61.00
ActualPrice	10692.0	3.564367e+01	148.246016	0.49	100.00
Volume	10692.0	8.473605e+02	664.309212	50.00	75000
TotalPurchaseQuantity	10692.0	3.140887e+03	11095.086769	1.00	36000
TotalPurchaseDollars	10692.0	3.010669e+04	123067.799627	0.71	430000
TotalSalesQuantity	10692.0	3.077482e+03	10952.851391	0.00	33000
TotalSalesDollars	10692.0	4.223907e+04	167655.265984	0.00	720000
TotalSalesPrice	10692.0	1.879378e+04	44952.773386	0.00	280000
TotalExciseTax	10692.0	1.774226e+03	10975.582240	0.00	43000
FreightCost	10692.0	6.143376e+04	60938.458032	0.09	140000
GrossProfit	10692.0	1.213238e+04	46224.337964	-52002.78	52000
ProfitMargin	10692.0	-inf	NaN	-inf	0.00
StockTurnover	10692.0	1.706793e+00	6.020460	0.00	0.00
SalestoPurchaseRatio	10692.0	2.504390e+00	8.459067	0.00	1.00

Distribution Plots for Numerical Columns

```
numerical_cols = df.select_dtypes(include = np.number).columns
```

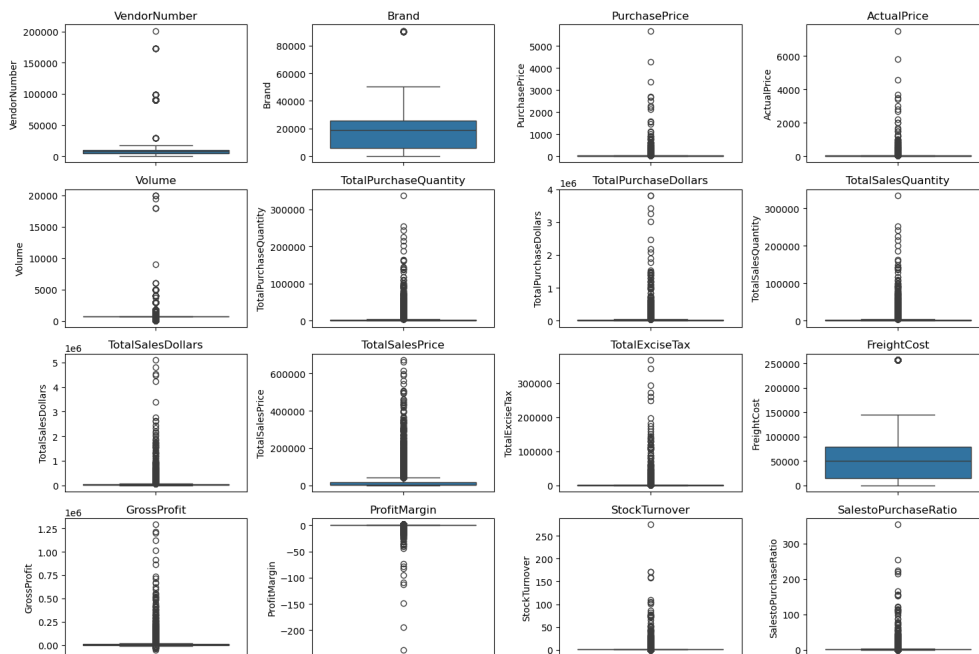
```
plt.figure(figsize=(15,10))
for i, col in enumerate(numerical_cols):
    plt.subplot(4,4, i+1)
    sns.histplot(df[col], kde = True, bins = 30)
    plt.title(col)
plt.tight_layout()
plt.show()
```



Outlier detection with boxplots

```
plt.figure(figsize = (15,10))
for i, col in enumerate(numerical_cols):
    plt.subplot(4,4,i+1)
```

```
sns.boxplot(y=df[col])
plt.title(col)
plt.tight_layout()
plt.show()
```



Summary Statistics Insights

Negative and Zero Values

- **Gross Profit:** The minimum value of -52,002.78 indicates losses, suggesting some products or transactions are being sold below cost—possibly due to heavy discounts or high procurement expenses.
- **Profit Margin:** A minimum value of negative infinity ($-\infty$) points to situations where revenue is zero or even lower than costs.
- **Total Sales Quantity & Sales Dollars:** Minimum values of 0 show that certain products were purchased but never sold, indicating slow-moving or obsolete inventory.

Outliers Reflected by High Standard Deviations

- **Purchase Price & Actual Price:** Maximum values (5,681.81 and 7,499.99) are far above their averages (24.39 and 35.64), highlighting the presence of high-end or premium products.
- **Freight Cost:** A wide range from 0.09 to 257,032.07 suggests inconsistencies in logistics, possibly due to bulk shipments or inefficient cost management.
- **Stock Turnover:** Values range from 0 to 274.5, indicating that some products sell very quickly while others remain unsold for long periods. A turnover greater than 1 implies that sales are being fulfilled using previously stocked inventory rather than recent purchases.

Let's filter the data by removing inconsistencies

```
df = pd.read_sql_query('''select * from vendor_sales_summary
where grossprofit > 0
and ProfitMargin > 0
and TotalSalesQuantity > 0''' , conn)
df
```

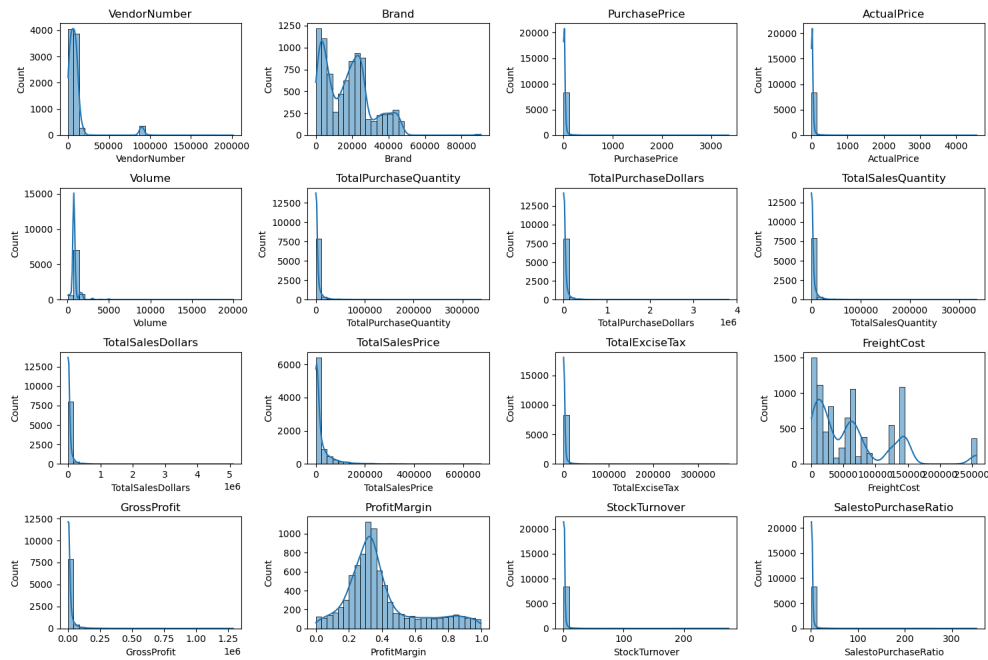
	VendorNumber	VendorName	Brand	Description	PurchasePrice
0	1128	BROWN-FORMAN CORP	1233	Jack Daniels No 7 Black	26.27
1	4425	MARTIGNETTI COMPANIES	3405	Tito's Handmade Vodka	23.19
2	17035	PERNOD RICARD USA	8068	Absolut 80 Proof	18.24
3	3960	DIAGEO NORTH AMERICA INC	4261	Capt Morgan Spiced Rum	16.17
4	3960	DIAGEO NORTH AMERICA INC	3545	Ketel One Vodka	21.89
...	...	...	...	...	...
8559	9815	WINE GROUP INC	8527	Concannon Glen Ellen Wh Zin	1.32
8560	8004	SAZERAC CO INC	5683	Dr McGillicuddy's Apple Pie	0.39
8561	3924	HEAVEN HILL DISTILLERIES	9123	Deep Eddy Vodka	0.74
8562	3960	DIAGEO NORTH AMERICA INC	6127	The Club Strawbry Margarita	1.47
8563	7245	PROXIMO SPIRITS INC.	3065	Three Olives Grape Vodka	0.71

8564 rows × 6 columns

Distribution Plots for Numerical Columns

```
numerical_cols = df.select_dtypes(include = np.number).columns
```

```
plt.figure(figsize=(15,10))
for i, col in enumerate(numerical_cols):
    plt.subplot(4,4, i+1)
    sns.histplot(df[col], kde = True, bins = 30)
    plt.title(col)
plt.tight_layout()
plt.show()
```



Count plots for categorical columns

```
categorical_cols = ['VendorName', 'Description']
```

```
plt.figure(figsize = (12,5))
```

```
for i, col in enumerate (categorical_cols):
```

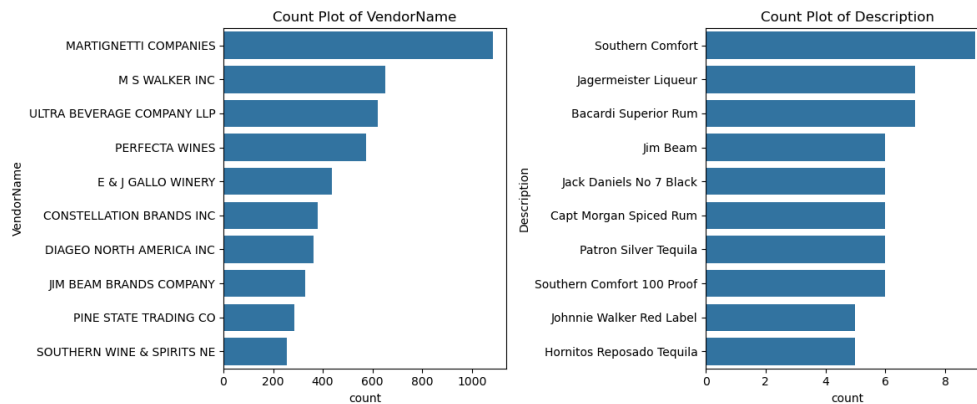
```
    plt.subplot(1,2, i+1)
```

```
    sns.countplot(y=df[col], order=df[col].value_counts().index[:10]) # top  
    10 categories
```

```
    plt.title(f'Count Plot of {col}')
```

```
plt.tight_layout()
```

```
plt.show()
```



Correlation Heatmap

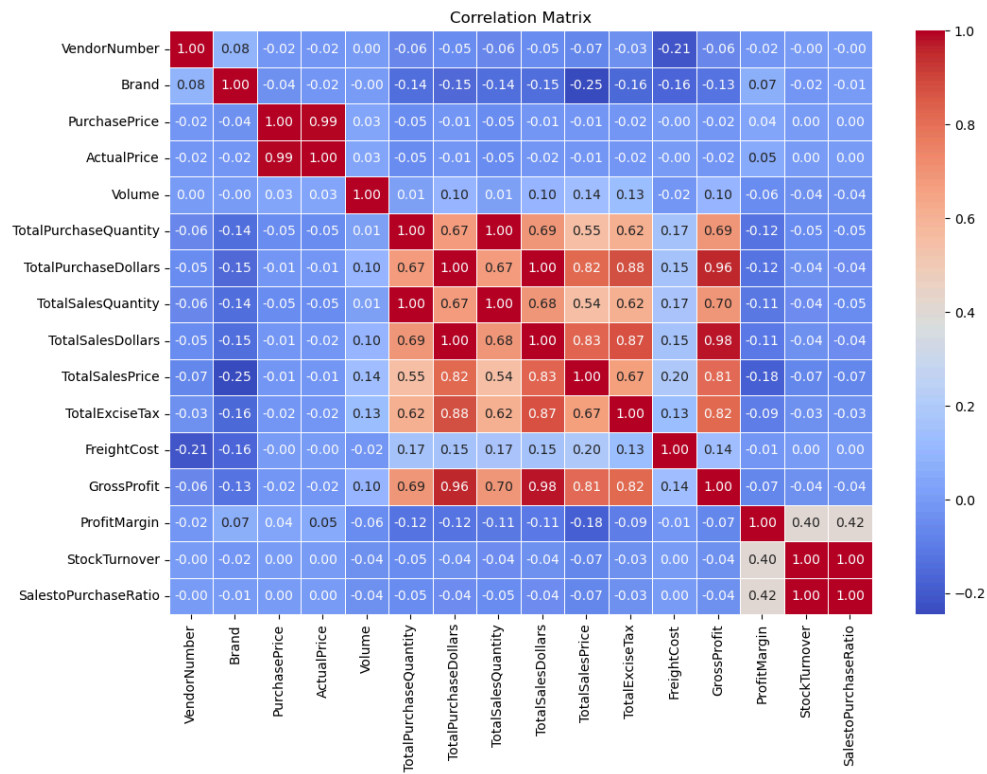
```
plt.figure(figsize=(12,8))
```

```
correlation_matrix = df[numerical_cols].corr()
```

```
sns.heatmap(correlation_matrix, annot = True, fmt='.2f', cmap='coolwarm',  
            linewidth = 0.5)
```

```
plt.title('Correlation Matrix')
```

```
plt.show()
```



Correlation Insights

- PurchasePrice has weak correlations with TotalSalesDollars (-0.012) and GrossProfit (-0.016), suggesting that price variations do not significantly impact sales revenue or profit.
- Strong correlation between total purchase quantity and total sales quantity (0.999), confirming efficient inventory turnover.
- Negative correlation between profit margin & total sales price (-0.179) suggests that as sales price increases, margins decrease, possibly due to competitive pricing pressures.
- StockTurnover has weak negative correlations with both GrossProfit (-0.038) and ProfitMargin (-0.055), indicating that faster turnover does not necessarily result in higher profitability.

Data Analysis

Identify Brands that needs Promotional or Pricing Adjustments which exhibit lower sales performance but higher profit margins.

```
brand_performance = df.groupby('Description').agg({
    'TotalSalesDollars': 'sum',
    'ProfitMargin': 'mean'}).reset_index()
```

brand_performance

	Description	TotalSalesDollars	ProfitMargin
0	(RI) 1	21519.09	0.180607
1	.nparalleled Svgn Blanc	1094.63	0.299782
2	10 Span Cab Svgn CC	2703.89	0.209376

	Description	TotalSalesDollars	ProfitMargin
3	10 Span Chard CC	3325.56	0.278064
4	10 Span Pnt Gris Monterey Cy	2082.22	0.322262
...	...	...	...
7702	Zorvino Vyds Sangiovese	10579.03	0.295257
7703	Zuccardi Q Malbec	1639.18	0.239815
7704	Zum Rsl	10857.34	0.326750
7705	Zwack Liqueur	227.88	0.166535
7706	von Buhl Jazz Rsl	1359.11	0.907734

7707 rows × 3 columns

```
low_sales_threshold =
    brand_performance['TotalSalesDollars'].astype(float).quantile(0.15)
high_margin_threshold =
    brand_performance['ProfitMargin'].astype(float).quantile(0.85)
```

low_sales_threshold

np.float64(560.299)

high_margin_threshold

np.float64(0.6497017552750112)

Filter brands with low sales but high profit margin

```
target_brands = brand_performance[
    (brand_performance['TotalSalesDollars'] <= low_sales_threshold) &
    (brand_performance['ProfitMargin'] >= high_margin_threshold)
]
```

```
print('Brands with low sales but high profit margins:')
display(target_brands.sort_values('TotalSalesDollars'))
```

Brands with low sales but high profit margins:

	Description	TotalSalesDollars	ProfitMargin
6199	Santa Rita Organic Svgn Bl	9.99	0.664665
2369	Debauchery Pnt Nr	11.58	0.659758
2070	Concannon Glen Ellen Wh Zin	15.95	0.834483
2188	Crown Royal Apple	27.86	0.898062
6237	Sauza Sprklg Wild Berry Marg	27.96	0.821531
...	...	...	...
5074	Nanbu Bijin Southern Beauty	535.68	0.767473
2271	Dad's Hat Rye Whiskey	538.89	0.818516
57	A Bichot Clos Marechaudes	539.94	0.677409
6245	Sbragia Home Ranch Merlot	549.75	0.664447
3326	Goulee Cos d'Estournal 10	558.87	0.694348

198 rows × 3 columns

brand_performance.dtypes

```

Description      object
TotalSalesDollars float64
ProfitMargin      float64
dtype: object

print(type(low_sales_threshold))
print(type(high_margin_threshold))

print(brand_performance['TotalSalesDollars'].dtype)
print(brand_performance['ProfitMargin'].dtype)

<class 'numpy.float64'>
<class 'numpy.float64'>
float64
float64

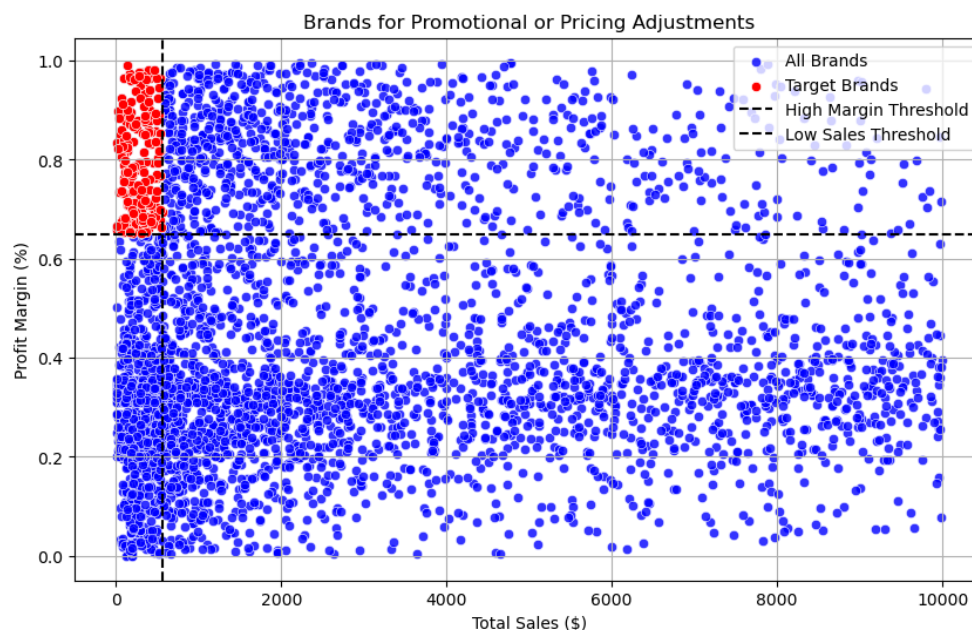
brand_performance = brand_performance[brand_performance['TotalSalesDollars']
                                       <10000]

plt.figure(figsize=(10,6))
sns.scatterplot(data=brand_performance, x='TotalSalesDollars',
                y='ProfitMargin', color = 'blue', label='All Brands', alpha=0.8)
sns.scatterplot(data=target_brands, x='TotalSalesDollars', y='ProfitMargin',
                color = 'red', label='Target Brands')

plt.axhline(high_margin_threshold, linestyle='--', color = 'black', label =
            'High Margin Threshold')
plt.axvline(low_sales_threshold, linestyle='--', color = 'black', label =
            'Low Sales Threshold')

plt.xlabel('Total Sales ($)')
plt.ylabel('Profit Margin (%)')
plt.title('Brands for Promotional or Pricing Adjustments')
plt.legend()
plt.grid(True)
plt.show()

```



Which vendors and brands demonstrate the highest sales performance?

```

def format_dollars(value):
    if value >= 1_000_000:

```

```

        return f'{value / 1_000_000:.2f}M'
    elif value >= 1_000:
        return f'{value / 1_000:.2f}K'
    else:
        return str(value)

```

```

top_vendors = df.groupby('VendorName')
                ['TotalSalesDollars'].sum().nlargest(10)
top_brands = df.groupby('Description')
                ['TotalSalesDollars'].sum().nlargest(10)
top_vendors

```

```

VendorName
DIAGEO NORTH AMERICA INC      67990099.42
MARTIGNETTI COMPANIES        39330359.36
PERNOD RICARD USA             32063196.19
JIM BEAM BRANDS COMPANY       31423020.46
BACARDI USA INC               24854817.14
CONSTELLATION BRANDS INC      24218745.65
E & J GALLO WINERY            18399899.46
BROWN-FORMAN CORP            18247230.65
ULTRA BEVERAGE COMPANY LLP    16502544.31
M S WALKER INC                14706458.51
Name: TotalSalesDollars, dtype: float64

```

```
top_brands
```

```

Description
Jack Daniels No 7 Black      7964746.76
Tito's Handmade Vodka        7399657.58
Grey Goose Vodka             7209608.06
Capt Morgan Spiced Rum      6356320.62
Absolut 80 Proof             6244752.03
Jameson Irish Whiskey        5715759.69
Ketel One Vodka              5070083.56
Baileys Irish Cream          4150122.07
Kahlua                       3604858.66
Tanqueray                    3456697.90
Name: TotalSalesDollars, dtype: float64

```

```
top_brands.apply(lambda x : format_dollars(x))
```

```

Description
Jack Daniels No 7 Black      7.96M
Tito's Handmade Vodka        7.40M
Grey Goose Vodka             7.21M
Capt Morgan Spiced Rum      6.36M
Absolut 80 Proof             6.24M
Jameson Irish Whiskey        5.72M
Ketel One Vodka              5.07M
Baileys Irish Cream          4.15M
Kahlua                       3.60M
Tanqueray                    3.46M
Name: TotalSalesDollars, dtype: object

```

```
top_vendors.apply(lambda x : format_dollars(x))
```

```

VendorName
DIAGEO NORTH AMERICA INC      67.99M
MARTIGNETTI COMPANIES        39.33M
PERNOD RICARD USA             32.06M
JIM BEAM BRANDS COMPANY       31.42M
BACARDI USA INC               24.85M

```

```

CONSTELLATION BRANDS INC      24.22M
E & J GALLO WINERY           18.40M
BROWN-FORMAN CORP           18.25M
ULTRA BEVERAGE COMPANY LLP   16.50M
M S WALKER INC               14.71M
Name: TotalSalesDollars, dtype: object

```

```
plt.figure(figsize=(15, 5))
```

```
# Plot for Top Vendors
```

```
plt.subplot(1, 2, 1)
```

```
ax1 = sns.barplot(y=top_vendors.index, x=top_vendors.values,
                  palette="Blues_r")
```

```
plt.title("Top 10 Vendors by Sales")
```

```
for bar in ax1.patches:
```

```

    ax1.text(bar.get_width() + (bar.get_width() * 0.02),
             bar.get_y() + bar.get_height() / 2,
             format_dollars(bar.get_width()),
             ha='left', va='center', fontsize=10, color='black')

```

```
# Plot for Top Brands
```

```
plt.subplot(1, 2, 2)
```

```
ax2 = sns.barplot(y=top_brands.index.astype(str), x=top_brands.values,
                  palette="Reds_r")
```

```
plt.title("Top 10 Brands by Sales")
```

```
for bar in ax2.patches:
```

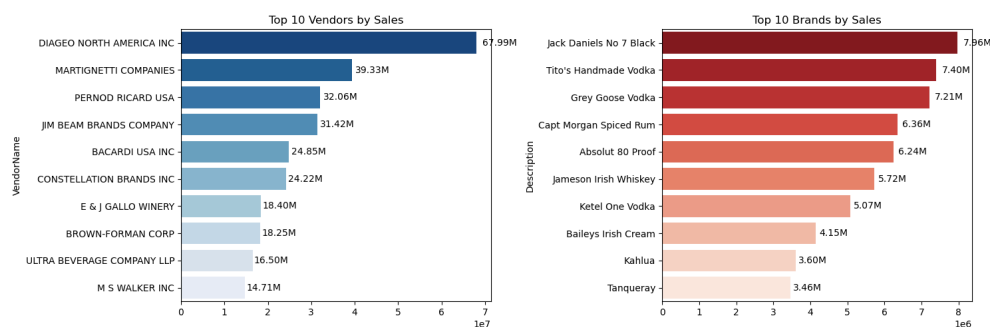
```

    ax2.text(bar.get_width() + (bar.get_width() * 0.02),
             bar.get_y() + bar.get_height() / 2,
             format_dollars(bar.get_width()),
             ha='left', va='center', fontsize=10, color='black')

```

```
plt.tight_layout()
```

```
plt.show()
```



Which vendors contribute the most to total purchase dollars?

```

vendor_performance = df.groupby('VendorName').agg({
    'TotalPurchaseDollars': 'sum',
    'GrossProfit' : 'sum',
    'TotalSalesDollars' : 'sum'
}).reset_index()

```

```

vendor_performance['PurchaseContribution%'] =
    vendor_performance['TotalPurchaseDollars']/vendor_performance['TotalPurchaseDollars'].s

```

```

vendor_performance =
    round(vendor_performance.sort_values('PurchaseContribution%',
    ascending = False),2)

```

```
# Display Top 10 Vendors
```

```
top_vendors = vendor_performance.head(10)
top_vendors['TotalSalesDollars'] =
    top_vendors['TotalSalesDollars'].apply(format_dollars)
top_vendors['TotalPurchaseDollars'] =
    top_vendors['TotalPurchaseDollars'].apply(format_dollars)
top_vendors['GrossProfit'] = top_vendors['GrossProfit'].apply(format_dollars)
top_vendors
```

	VendorName	TotalPurchaseDollars	GrossProfit	TotalSalesDollars
25	DIAGEO NORTH AMERICA INC	50.10M	17.89M	67.99M
57	MARTIGNETTI COMPANIES	25.50M	13.83M	39.33M
68	PERNOD RICARD USA	23.85M	8.21M	32.06M
46	JIM BEAM BRANDS COMPANY	23.49M	7.93M	31.42M
6	BACARDI USA INC	17.43M	7.42M	24.85M
20	CONSTELLATION BRANDS INC	15.27M	8.95M	24.22M
11	BROWN-FORMAN CORP	13.24M	5.01M	18.25M
30	E & J GALLO WINERY	12.07M	6.33M	18.40M
106	ULTRA BEVERAGE COMPANY LLP	11.17M	5.34M	16.50M
53	M S WALKER INC	9.76M	4.94M	14.71M

```
top_vendors['CumulativeContribution%'] =
    top_vendors['PurchaseContribution%'].cumsum()
```

```
top_vendors
```

	VendorName	TotalPurchaseDollars	GrossProfit	TotalSalesDollars
25	DIAGEO NORTH AMERICA INC	50.10M	17.89M	67.99M
57	MARTIGNETTI COMPANIES	25.50M	13.83M	39.33M
68	PERNOD RICARD USA	23.85M	8.21M	32.06M
46	JIM BEAM BRANDS COMPANY	23.49M	7.93M	31.42M
6	BACARDI USA INC	17.43M	7.42M	24.85M
20	CONSTELLATION BRANDS INC	15.27M	8.95M	24.22M
11	BROWN-FORMAN CORP	13.24M	5.01M	18.25M
30	E & J GALLO WINERY	12.07M	6.33M	18.40M

	VendorName	TotalPurchaseDollars	GrossProfit	TotalSalesDollars
106	ULTRA BEVERAGE COMPANY LLP	11.17M	5.34M	16.50M
53	M S WALKER INC	9.76M	4.94M	14.71M

How much of total procurement is dependent on the top vendors?

```
import matplotlib.pyplot as plt

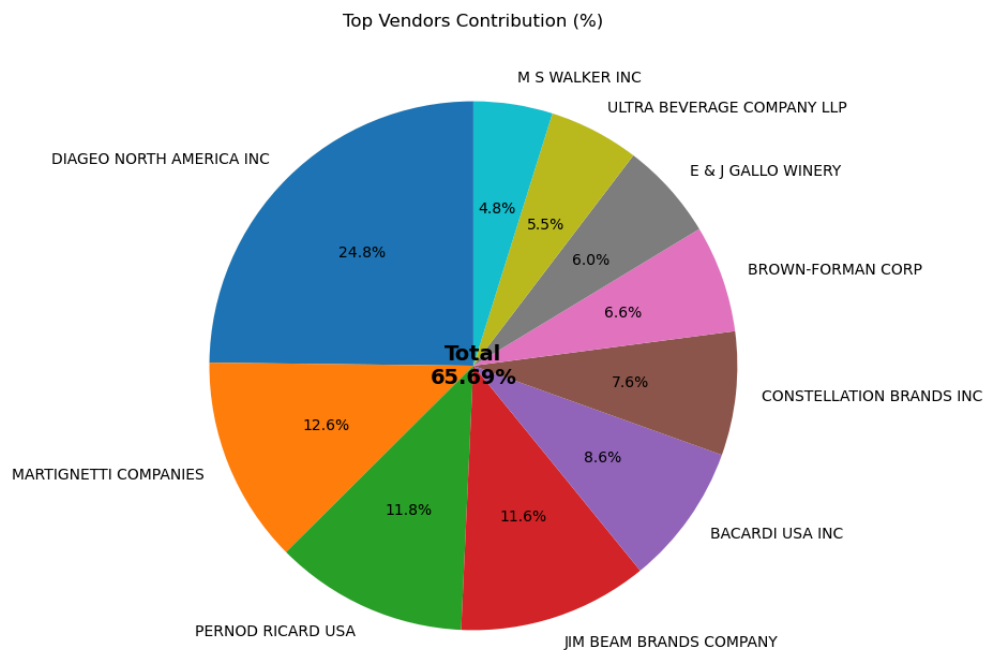
total_pct = top_vendors['PurchaseContribution%'].sum()

plt.figure(figsize=(8,8))

plt.pie(
    top_vendors['PurchaseContribution%'],
    labels=top_vendors['VendorName'],
    autopct='%1.1f%%',
    startangle=90
)

plt.text(0, 0, f'Total\n{total_pct:.2f}%',
        ha='center', va='center', fontsize=14, fontweight='bold')

plt.title('Top Vendors Contribution (%)')
plt.show()
```



The remaining vendors contribute only 34.31%, meaning they are not utilized effectively or may not be as competitive. If vendor dependency is too high, consider identifying new suppliers to reduce risk.

Does purchasing in bulk reduce the unit price, and what is the optimal purchase volume for cost savings?

```
df['UnitPurchasePrice'] = df['TotalPurchaseDollars'] /
    df['TotalPurchaseQuantity']
```

df

	VendorNumber	VendorName	Brand	Description	PurchasePrice
0	1128	BROWN-FORMAN CORP	1233	Jack Daniels No 7 Black	26.27
1	4425	MARTIGNETTI COMPANIES	3405	Tito's Handmade Vodka	23.19
2	17035	PERNOD RICARD USA	8068	Absolut 80 Proof	18.24
3	3960	DIAGEO NORTH AMERICA INC	4261	Capt Morgan Spiced Rum	16.17
4	3960	DIAGEO NORTH AMERICA INC	3545	Ketel One Vodka	21.89
...	...	...	...	...	...
8559	9815	WINE GROUP INC	8527	Concannon Glen Ellen Wh Zin	1.32
8560	8004	SAZERAC CO INC	5683	Dr McGillicuddy's Apple Pie	0.39
8561	3924	HEAVEN HILL DISTILLERIES	9123	Deep Eddy Vodka	0.74
8562	3960	DIAGEO NORTH AMERICA INC	6127	The Club Strawbry Margarita	1.47
8563	7245	PROXIMO SPIRITS INC.	3065	Three Olives Grape Vodka	0.71

8564 rows × 19 columns

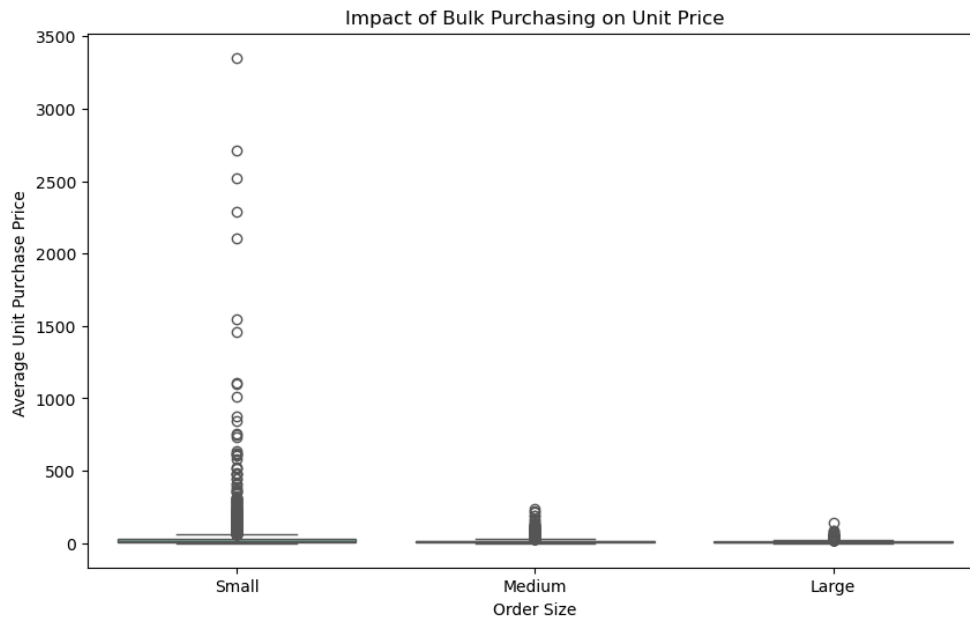
```
df['OrderSize'] = pd.qcut(df['TotalPurchaseQuantity'], q=3, labels =
    ['Small', 'Medium', 'Large'])
```

```
df.groupby('OrderSize')[['UnitPurchasePrice']].mean()
```

	UnitPurchasePrice
OrderSize	
Small	39.068186
Medium	15.486414
Large	10.777625

```
plt.figure(figsize=(10,6))
sns.boxplot(data=df, x='OrderSize', y='UnitPurchasePrice', palette='Set2')
plt.title('Impact of Bulk Purchasing on Unit Price')
plt.xlabel('Order Size')
```

```
plt.ylabel('Average Unit Purchase Price ')
plt.show()
```



- Vendors buying in bulk (Large Order Size) get the lowest unit price (\$10.78 per unit), meaning higher margins if they can manage inventory efficiently.
- The price difference between Small and Large orders is substantial (~72% reduction in unit cost)
- This suggests that bulk pricing strategies successfully encourage vendors to purchase in larger volumes, leading to higher overall sales despite lower per-unit revenue.

Which vendors have low inventory turnover, indicating excess stock and slow-moving products?

```
df[df['StockTurnover']<1].groupby('VendorName')
  [['StockTurnover']].mean().sort_values('StockTurnover', ascending =
    True).head(10)
```

	StockTurnover	
VendorName		
ALISA CARR BEVERAGES	0.615385	
HIGHLAND WINE MERCHANTS LLC	0.708333	
PARK STREET IMPORTS LLC	0.751306	
Circa Wines	0.755676	
Dunn Wine Brokers	0.766022	
CENTEUR IMPORTS LLC	0.773953	
SMOKY QUARTZ DISTILLERY LLC	0.783835	
TAMWORTH DISTILLING	0.797078	
THE IMPORTED GRAPE LLC	0.807569	
WALPOLE MTN VIEW WINERY	0.820548	

How much capital is locked in unsold inventory per vendor, and which vendors contribute the most to it?

```
df['UnsoldInventoryValue'] = (df['TotalPurchaseQuantity'] -
                             df['TotalSalesQuantity'])*df['PurchasePrice']
print('Total Unsold Capital:',
      format_dollars(df['UnsoldInventoryValue'].sum()))
```

Total Unsold Capital: 2.71M

Aggregate Capital Locked per vendor

```
inventory_value_per_vendor = df.groupby('VendorName')
                             ['UnsoldInventoryValue'].sum().reset_index()
```

Sort vendors with highest Locked capital

```
inventory_value_per_vendor = inventory_value_per_vendor.sort_values(by =
                             'UnsoldInventoryValue', ascending = False)
inventory_value_per_vendor['UnsoldInventoryValue'] =
    inventory_value_per_vendor['UnsoldInventoryValue'].apply(format_dollars)
inventory_value_per_vendor.head(10)
```

	VendorName	UnsoldInventoryValue
25	DIAGEO NORTH AMERICA INC	722.21K
46	JIM BEAM BRANDS COMPANY	554.67K
68	PERNOD RICARD USA	470.63K
116	WILLIAM GRANT & SONS INC	401.96K
30	E & J GALLO WINERY	228.28K
79	SAZERAC CO INC	198.44K
11	BROWN-FORMAN CORP	177.73K
20	CONSTELLATION BRANDS INC	133.62K
61	MOET HENNESSY USA INC	126.48K
77	REMY COINTREAU USA INC	118.60K